

Tutorial - Semana 1, Encontro 2

Introdução

No Encontro 1, o projeto do Vertical Slice foi criado e organizado, mas ainda não existe nenhum conteúdo dentro dele. Este encontro resolve o próximo problema universal de qualquer engine: como compor um objeto de jogo a partir de partes menores. O Godot resolve isso através de **Node** e **Scene Tree**, usando o **Orchestrator** como ferramenta de construção visual.

Este tutorial dá continuidade direta ao Encontro 1 — a estrutura de pastas criada anteriormente já deve existir no projeto.

Objetivos da semana

- Explicar Node e Scene Tree como unidade universal de composição.
- Comparar composição via Node (Godot) com composição via Component (Unity).
- Criar uma Scene com Nodes filhos usando o Orchestrator.

Resultado esperado ao final da semana

Ao final deste encontro, cada estudante terá uma Scene funcional com ao menos três Nodes organizados em hierarquia, incluindo um Node não demonstrado em aula, salva dentro da estrutura de pastas do projeto já criada.

Pré-requisitos

- Projeto do Vertical Slice criado e estruturado (ver Tutorial - Semana 1, Encontro 1).
 - Addon Orchestrator instalado e habilitado no projeto.
-

Antes de começar

O que o estudante deverá possuir antes desta semana

- O projeto Godot criado no Encontro 1, com a estrutura completa de pastas (`scenes/` , `scripts/` , `orchestrations/` , `resources/` , `assets/` , `materials/` , `audio/` , `animations/`).

Arquivos necessários

- Nenhum arquivo externo. Todos os Nodes desta atividade são criados diretamente no editor, sem assets de terceiros.

Assets utilizados

- Nenhum. Os pacotes Kenney entram em cena a partir da Semana 3.

Projeto esperado

- Projeto aberto no Godot 4.7, com o addon Orchestrator habilitado em **Project > Project Settings > Plugins**.

Imagem sugerida

Objetivo: mostrar a aba Plugins do Project Settings com o Orchestrator listado e habilitado.

Enquadramento: captura de tela da janela Project Settings, aba Plugins. Elementos

importantes: nome "Orchestrator" na lista, checkbox de status marcado como habilitado.

Destaque: o checkbox de ativação do plugin. Legenda sugerida: "Orchestrator habilitado como plugin no Project Settings."

Parte 1 — Composição versus herança: Node e Scene Tree

Objetivo

Entender por que o Godot organiza um objeto de jogo como uma árvore de Nodes, e não como uma classe que herda comportamento de outra.

Conceito

Toda engine moderna precisa resolver o mesmo problema: como montar um objeto de jogo (um personagem, uma porta, uma luz) a partir de partes menores e reutilizáveis. Existem duas estratégias gerais para isso:

- **Herança:** uma classe herda comportamento de uma classe-base (ex.: `Inimigo` herda de `Personagem`).
- **Composição:** um objeto reúne partes independentes, cada uma responsável por um pedaço do comportamento (ex.: um objeto tem um componente de vida, um componente de movimento, um componente de renderização).

O Godot resolve esse problema através de **Node** e **Scene Tree**: uma **Scene** é uma árvore de Nodes, e cada Node especializado adicionado como filho de outro é, na prática, um componente de comportamento daquele objeto. Um `Node3D` com uma `MeshInstance3D` e uma `Light3D` como filhos não é "um objeto com propriedades de malha e luz" — é uma composição de três Nodes independentes, cada um responsável por sua própria função.

Esse é o mesmo princípio de composição por trás do par `GameObject/Component` da Unity. A diferença está em *como* cada engine implementa essa árvore, não se a composição existe.

Passo a passo

1. Sem abrir o Godot ainda, desenhe (no quadro ou papel) uma árvore simples: um nó pai com dois nós filhos, cada um com um rótulo de função (ex.: "posição", "aparência", "luz").
2. Compare essa árvore com a estrutura de um `GameObject` da Unity com dois `Components` anexados.
3. Identifique, na dupla ou com a turma, a diferença: no diagrama do Godot, os "componentes" são eles mesmos nós da árvore; na Unity, os `Components` vivem "dentro" de um `GameObject` contêiner.

Resultado esperado

A turma consegue desenhar ou descrever verbalmente a diferença estrutural entre a Scene Tree do Godot e o par GameObject/Component da Unity, sem ainda ter aberto o editor.

Verificando

Peça para um estudante explicar, sem usar o termo "GameObject", o que é um Node dentro de uma Scene Tree.

Problemas comuns

- Achar que "Node" e "Component" são termos intercambiáveis sem nuance — reforçar que no Godot o próprio componente é um nó da árvore, e não um item anexado a um contêiner vazio.

Boas práticas

- Sempre pensar em uma Scene como uma árvore de responsabilidades, não como uma lista plana de objetos.

Comparação com Unity

Na Unity, um GameObject nasce vazio e ganha comportamento ao receber Components (Transform, Rigidbody, um script, etc.). No Godot, cada Node já nasce especializado (Node3D, MeshInstance3D, Light3D, CharacterBody3D, entre outros), e a composição acontece organizando esses Nodes especializados como filhos dentro de uma Scene. O resultado final é parecido — um objeto com múltiplas capacidades —, mas o modelo mental de construção é diferente.

Parte 2 — Criação guiada de uma Scene com Nodes filhos via Orchestrator

Objetivo

Criar a primeira Scene do Vertical Slice, com uma pequena hierarquia de Nodes, utilizando o Orchestrator como camada de construção visual.

Conceito

O Orchestrator é a ferramenta de visual scripting adotada como camada principal de scripting da disciplina, cumprindo papel equivalente ao Blueprint da Unreal Engine. Ele permite criar e conectar lógica de forma visual, sem escrever GDScript diretamente — o GDScript entra apenas como apoio, quando o Orchestrator não cobrir algum recurso específico (situação que não ocorre neste primeiro encontro).

Uma Scene no Godot é salva como um único arquivo `.tscn`, que descreve a árvore de Nodes daquele contexto. Uma Scene pode representar um personagem, um objeto do cenário, uma tela de UI ou um nível inteiro — a diferença está apenas na composição dos Nodes dentro dela.

Passo a passo

1. No FileSystem Dock, clique com o botão direito na pasta `scenes/levels/exploration` e selecione **New Scene**.
2. Na janela de criação de Scene, escolha **Node3D** como tipo do nó raiz — ele representa um ponto de organização espacial vazio, sem malha nem comportamento próprios.
3. Renomeie o nó raiz para `NívelTeste` (nome descritivo, seguindo a convenção PascalCase de Scenes).
4. Clique com o botão direito sobre `NívelTeste` na árvore de cena e selecione **Add Child Node**.
5. Adicione um Node do tipo **MeshInstance3D** como filho, e renomeie-o para `Chao`.
6. No painel Inspector, com `Chao` selecionado, defina uma **Mesh** simples (ex.: um `BoxMesh` ou `PlaneMesh`) para que ele fique visível no Viewport.
7. Adicione um segundo Node filho do tipo **DirectionalLight3D**, renomeando-o para `LuzPrincipal`.
8. Confirme, no Viewport 3D, que o objeto `Chao` está visível e iluminado pela `LuzPrincipal`.
9. Abra o painel do **Orchestrator** (geralmente disponível como aba adicional próxima ao editor de Script) e crie uma nova Orchestration associada ao Node raiz `NívelTeste`, salvando-a em

`orchestrations/` com o nome `nivel_teste.os`.

10. Dentro da Orchestration, adicione um nó de evento **Ready** (equivalente ao `_ready()` do GDScript) conectado a um nó simples de saída de log/print, apenas para confirmar que a Orchestration está ativa e conectada à Scene.
11. Salve a Scene (**Ctrl+S**) com o nome `level_exploration.tscn`, seguindo a convenção de nomenclatura de cena de nível.
12. Pressione **Play Scene** (ou F6) para testar a Scene isoladamente e confirmar que ela abre sem erros.

Resultado esperado

A Scene `level_exploration.tscn` existe dentro de `scenes/levels/exploration/`, com uma hierarquia de três Nodes (`NivelTeste` > `Chao`, `LuzPrincipal`), uma Orchestration associada funcional, e roda sem erros ao ser testada isoladamente.

Verificando

1. Confirme, na árvore de cena, que `Chao` e `LuzPrincipal` são filhos diretos de `NivelTeste`.
2. Rode a Scene com F6 e confirme que nenhuma janela de erro aparece.
3. Verifique, no painel de saída (Output), que a mensagem de log configurada no evento Ready da Orchestration aparece ao rodar a cena.

Problemas comuns

- `Chao` invisível no Viewport: verificar se uma Mesh foi de fato atribuída no Inspector — um `MeshInstance3D` sem Mesh definida não renderiza nada.
- Orchestration não executa: confirmar que ela está corretamente associada ao Node `NivelTeste` (não a outro Node da hierarquia) e que o addon Orchestrator está habilitado no Project Settings.
- Erro ao testar a Scene isolada pedindo uma "Main Scene": esse erro é esperado neste momento (nenhuma Main Scene foi definida ainda) e não impede o teste via F6 diretamente na Scene aberta.

Boas práticas

- Nunca deixar um Node com o nome padrão gerado pelo editor (`Node3D`, `MeshInstance3D2`) — sempre renomear para um nome que descreva a função, conforme a convenção de nomenclatura do projeto (PROJECT_ARCHITECTURE.md, seção 9).

- Manter a Orchestration organizada desde o início, evitando grafos extensos e pouco legíveis — mesmo em uma Orchestration simples como esta.

Comparação com Unity

Criar esta mesma hierarquia na Unity envolveria criar um `GameObject` vazio, anexar um Component de malha (`MeshFilter` + `MeshRenderer`) e um Component de luz (`Light`) a `GameObjects` filhos, e anexar um script C# (ou usar Visual Scripting da própria Unity) ao `GameObject` pai para lógica equivalente ao evento Ready. O resultado funcional é semelhante, mas no Godot cada peça (malha, luz) já é, por si só, um Node completo dentro da árvore, sem a etapa intermediária de "criar um contêiner vazio e anexar componentes a ele".

Ao final da semana

Ao final da Semana 1 (Encontros 1 e 2), o projeto do Vertical Slice deve conter:

- A estrutura de pastas completa, criada no Encontro 1.
- A primeira Scene funcional do projeto (`level_exploration.tscn`), com uma hierarquia de Nodes organizada e uma Orchestration associada.
- Um Node adicional, criado de forma autônoma no desafio deste encontro, ainda não presente na demonstração do professor.

Segundo o `PROJECT_ARCHITECTURE.md` (seção 6, Módulo 1), este resultado corresponde ao item "Node base + composição", pré-requisito direto para o próximo passo do roadmap: o Player (`CharacterBody3D`) com locomoção, que será construído na Semana 2. Nenhum conteúdo desta semana será refeito — apenas ampliado.

Desafio

Adicione um Node filho adicional à Scene `level_exploration.tscn`, não demonstrado neste tutorial, produzindo um comportamento visual diferente do exemplo do professor (por exemplo, um segundo `MeshInstance3D` com outra forma, ou uma segunda fonte de luz do tipo `OmniLight3D`). A escolha do Node é livre, desde que compatível com o escopo do Vertical Slice descrito no `PROJECT_ARCHITECTURE.md` — não introduza nenhum sistema fora do escopo (seção 13 do documento).

Checklist

- ☐ Scene `level_exploration.tscn` criada em `scenes/levels/exploration/`
- ☐ Hierarquia com `NivelTeste` (Node3D) como raiz, `Chao` (MeshInstance3D) e `LuzPrincipal` (DirectionalLight3D) como filhos
- ☐ Todos os Nodes renomeados de forma descritiva (nenhum nome padrão do editor)
- ☐ Orchestration `nivel_teste.os` associada à Scene e funcional (evento Ready testado)
- ☐ Scene testada com F6 sem erros
- ☐ Node adicional do desafio criado e distinto do exemplo demonstrado

Glossário

- **Node:** unidade básica de composição no Godot; cada Node é especializado em uma função (malha, luz, física, script).
- **Scene:** árvore de Nodes salva em um arquivo `.tscn`, podendo representar um personagem, objeto ou nível.
- **Scene Tree:** a hierarquia de Nodes que compõe uma Scene em execução.
- **Orchestrator:** addon de visual scripting adotado como camada principal de lógica na disciplina, equivalente ao Blueprint da Unreal Engine.
- **Composição:** estratégia de construir um objeto a partir de partes independentes, em oposição à herança de classes.

Referências

- Godot Documentation — Nodes and Scenes: https://docs.godotengine.org/en/stable/tutorials/scripting/nodes_and_scene_instances.html
- Godot Documentation — Getting Started / Introduction: https://docs.godotengine.org/en/stable/getting_started/introduction/index.html
- Orchestrator — Documentação oficial: <https://orchestrator.cratercrash.space/>
- GDQuest: <https://www.gdquest.com/>
- Unity Manual (consulta comparativa): <https://docs.unity3d.com/Manual/>